# An Experimental Pipeline for Human-Centered Evaluation of XAI-Assisted Decision Making

Improving the Blockies User Study

Likhith Kumar Shivakumar
Sai Sura
Indra Karan Nukala

Bielefeld University

# Contents

**1 Introduction** **3**
1.1 Problem and motivation . . . 3
1.2 The Blockies framework . . . 3
1.3 Project goals and requirements . . . 3
1.4 Contributions . . . 3
1.5 Outline of this report . . . 4

**2 Related work** **4**
2.1 Human-centered evaluation of XAI . . . 4
2.2 Proxy tasks, application-grounded studies and synthetic data . . . 5
2.3 Reliance on AI advice . . . 5
2.4 Software for user studies: design options . . . 5
2.5 Summary . . . 5

**3 Methodology** **6**
3.1 Scope and pipeline . . . 6
3.2 Starting point . . . 6
3.3 Architecture of the refactored app . . . 7
3.4 The shared single-page client (R1, R2, R6, R7) . . . 7
3.5 Web-based study creation (R3) . . . 8
3.6 Automatic layout generation (R6, R7) . . . 9
3.7 Integration with the Blockies dataset generator . . . 10
3.8 Data integrity (R4) . . . 10
3.9 Testing framework (R5) . . . 10
3.10 Interface and interaction . . . 11
3.11 Deployment with Docker . . . 11
3.12 Problems found during development . . . 11

**4 Experiments** **13**
4.1 Experimental setup . . . 13
4.2 Evaluation procedure and metrics . . . 14
4.3 Results . . . 14
4.4 Analysis . . . 15
4.5 Limitations of the evaluation . . . 16

**5 Conclusion and summary** **16**
5.1 Summary of the project . . . 16
5.2 Results . . . 16
5.3 Outlook . . . 16
5.4 Availability . . . 17

**References** **17**

**Abstract**

Human-centered evaluation of explainable AI (XAI) needs controlled user studies, and running such studies is slow when every step of the workflow is manual. This report describes our work on the Blockies experimental framework, which combines a synthetic dataset generator, AI and XAI models, and a web application in which participants complete AI-supported diagnostic tasks. We focused on the first phase of the project: the structure, maintainability and reliability of the User Study Web App, and its connection to the dataset generator. We replaced the duplicated per-study HTML, CSS and JavaScript files with one shared single-page client that builds each study from a configuration file. We added a browser form that creates a new study from a CSV file and a ZIP archive of images. A layout generator reads the CSV columns and chooses the right interface cards automatically. We protected the stored participant data with input validation, a unique index and automatic timestamps, and we added an automated backend test suite with Mocha, Chai and Supertest. We also fixed the Docker build so that the app starts in a container. All seven project requirements (R1 to R7) are met. Studies of several types produced by the generator were created and rendered without manual edits to the configuration. The result is a smoother, more reproducible path from generated data to a running user study. We close with limitations and the next steps: connecting AI and XAI model outputs directly, evaluating a component framework such as React with TypeScript and Vite, and adding interface tests. **Keywords:** explainable AI, user studies, human-AI decision-making, experimental pipeline, web application, reproducibility

# 1 Introduction

## 1.1 Problem and motivation

Artificial intelligence is increasingly used to support decisions in high-stakes areas such as medicine. The hope is that people and AI systems together decide better than either alone. A recent meta-analysis found that this benefit has not yet been fully realised, because human-AI teams often perform worse than either humans or AI alone (Vaccaro et al., 2024). Explainable AI (XAI) is one proposed remedy: explanations should help people judge when to trust an AI recommendation and when to question it. Whether explanations really help, and in which form, is an empirical question. It can only be answered with user studies in which real people work with AI advice.

Such studies are expensive to build. Studies with domain experts are realistic but costly and hard to scale. Studies with proxy tasks are cheap, but their results may not predict real-world outcomes (Buçinca et al., 2020). Both kinds also tend to be assessed in different ways, which limits reproducibility and comparison across studies. A further practical problem is the software itself. Each new study needs its own interface, its own data, its own configuration and its own way of storing results. When these steps are done by hand, they are slow and error-prone, and mistakes in the software can silently damage the collected data.

This report is about that practical problem. It describes how we improved an existing experimental framework so that a new user study can be created quickly, reproducibly and with confidence that the results are stored correctly.

## 1.2 The Blockies framework

Blockies is a parametric dataset generator for visual diagnostic tasks (Johnson, 2026). In a Blockies task, a participant diagnoses virtual 3D creatures, called Blockies, from observed traits and symptoms, shown as so-called X-rays. The generator gives fine control over task difficulty, bias and ambiguity, and it is suitable for online studies with general users. The framework has three components:

1. **The dataset generator** (blockies-haic), which produces images and metadata.
2. **AI and XAI model development** (blockies-xai), which trains a model and prepares AI advice and explanations.
3. **The User Study Web App** (xai-eval-app), in which participants complete the diagnostic task with the AI support and their answers are stored.

Together these components form an experimental pipeline for human-centered evaluation of XAI-assisted decision making: data are generated, models and explanations are prepared, a study is configured, participants take part, and the results are analysed. At the start of our project the three components were largely independent. Moving from one stage to the next required substantial manual work: transferring generated datasets, preparing AI and XAI outputs for the study interface, and configuring the web app for each experimental condition.

## 1.3 Project goals and requirements

The goal of the project is to make this workflow smoother, more reproducible and less error-prone by reducing manual steps and improving the integration between the components. The first phase, which this report covers, concentrates on the interface component: the structure and maintainability of the User Study Web App. At the start, the web app contained significant code duplication across experimental conditions and study IDs. It arose from the way new studies were created: the HTML, CSS and JSON files were copied from a template folder into each new study folder. The project brief set seven requirements, summarised in Table 1.

## 1.4 Contributions

Our contributions are the following.

| ID | Requirement | In short |
|---|---|---|
| R1 | Maintainability | Refactor so shared components are updated in one place |
| R2 | Reusable functionality | New task, advice, XAI, timing and logging features are reusable across studies |
| R3 | Study creation | Create a study from a CSV metadata file and a ZIP archive of images |
| R4 | Data accuracy | Participant results are stored accurately in the database |
| R5 | Testing framework | Automated tests verify that results are recorded correctly |
| R6 | Componentized interface | Shared cards, a header with progress bar, and a footer with navigation |
| R7 | Configurable layouts | Layouts are defined by configuration, not hard-coded pages |

**Table 1:** The seven requirements of the project brief for the User Study Web App.

- A **single-page client** shared by all studies. Each study keeps only a configuration file and its images (R1, R2, R6, R7).
- A **web-based study creation** workflow that turns a CSV file and a ZIP archive into a ready study (R3).
- An **automatic layout generator** that reads the CSV columns and chooses the interface cards, so that the output of the Blockies generator can be uploaded without manual editing (R6, R7).
- **Data integrity measures**: input validation, a unique index and automatic timestamps (R4).
- An **automated test suite** for the backend with Mocha, Chai and Supertest (R5).
- A **Docker deployment** that builds and starts reliably, and a set of interface improvements that make the app responsive and easier to use.

### 1.5 Outline of this report

Section 2 reviews related work on the evaluation of XAI, on reliance in human-AI decision making, and on the design options for study software. Section 3 explains our methodology: the architecture, the shared client, study creation, layout generation, data integrity, testing and deployment. Section 4 describes the experimental setup, the evaluation procedure and metrics, and the results. Section 5 summarises the project and gives an outlook.

## 2 Related work

The problem in this report has two sides. The first is scientific: how to evaluate XAI with people. The second is practical: which software can run such evaluations reliably. This section reviews the literature on both sides and ends with the design options that we considered. The works cited are those listed in the Blockies paper (Johnson, 2026) and the studies it builds on; we did not carry out a full systematic review.

### 2.1 Human-centered evaluation of XAI

Doshi-Velez and Kim (2017) argued that interpretability needs a rigorous science of evaluation. They separate evaluation that uses real tasks with domain experts (application-grounded), evaluation that uses simplified tasks with lay people (human-grounded), and evaluation that uses formal proxies without people (functionally grounded). Systematic reviews of the field survey how XAI is evaluated in clinical

decision support, where a human-centered view is especially needed (Antoniadi et al., 2021). A common finding is that human evaluation is rare. Suh et al. (2025) report that fewer than 1% of XAI papers validate explainability with humans.

The empirical results that do exist are mixed. Explanations can reduce human-AI performance by inducing mistrust, through both overreliance (Buçinca et al., 2021) and underreliance (Vasconcelos et al., 2023). Together, these works show that conclusions depend strongly on the task, the participants and the study design. That is why a reusable, well-controlled study infrastructure is useful.

## 2.2 Proxy tasks, application-grounded studies and synthetic data

Many evaluations use proxy tasks, for example asking people to predict a model's behaviour. Buçinca et al. (2020) show that proxy tasks and subjective measures can be misleading when evaluating XAI systems. Application-grounded evaluation with real tasks and experts is considered the gold standard (Doshi-Velez and Kim, 2017). It is often too costly and slow to scale. Online application-grounded studies exist, but their assessments were heterogeneous, which limits reproducibility and comparison between studies (Johnson, 2026).

Synthetic datasets offer a middle path. Two4Two (Sixt et al., 2022) generates images of abstract animals so that the relevant features are known exactly, and it comes with a user study and a baseline. Blockies extends this idea. Instead of one trait that determines the class, diagnostic classes are defined by sets of symptoms, and the generator controls task complexity, bias and ambiguity (Johnson, 2026). Such a dataset can be used by general users online, without domain experts, while keeping the ground truth under the experimenter's control. Our project does not change this data. It concerns the software that turns generated data into a running study.

## 2.3 Reliance on AI advice

The central outcome of many human-AI studies is how much people rely on AI advice. Overreliance means following wrong AI advice, and underreliance means ignoring correct advice. Cognitive forcing functions can reduce overreliance (Buçinca et al., 2021). Schemmer et al. (2023) define appropriate reliance as a framework that focuses on cases where the human's initial decision differs from the AI. The Blockies study uses this framework to compute relative AI reliance and relative self reliance (Johnson, 2026).

Stakes are a further factor. Research suggests that perceived consequences influence how a user chooses to rely on AI (Starcke and Brand, 2012; Vasconcelos et al., 2023). The Blockies user study, which is our use case, examined raised versus low stakes and found longer deliberation but less calibrated reliance under raised stakes. For our work, the important point is the requirement this places on the software: timing, decisions and condition-specific content (such as a warning shown only in one condition) must be recorded exactly, so that such analyses are valid.

## 2.4 Software for user studies: design options

Research on study design says little about how the study software should be built, so we compared four approaches that could meet the project requirements.

Approach A is the situation we started from. Approach B was not considered further because the studies need exact control over the diagnostic layout and over what is logged. Approach C matches the suggestion in the project brief and remains attractive. We chose approach D for this phase because it removes the duplication, keeps the existing server and database, and can later be moved into a component framework, since the configuration format already separates content from code.

## 2.5 Summary

The literature shows that human-centered evaluation of XAI is needed, that results depend on task and design, and that reliance and stakes are central. It also shows a gap on the software side: an experimental

| Approach | Strengths | Limits for this project |
|---|---|---|
| A. One copy of the interface per study (status quo) | Simple; each study is independent | Duplicated code; changes must be repeated; risk of inconsistent studies (against R1) |
| B. General-purpose survey or experiment builder | Quick setup; no own code | Hard to show AI advice and XAI displays with controlled layouts and exact logging (against R2, R4, R7) |
| C. Component framework such as React with TypeScript and Vite | Strong reuse and structure; suggested in the project brief | Larger rewrite; needs a build process; a good later step (R6) |
| D. Shared single-page client driven by a configuration file | Small change to the existing app; code once, data per study | Layout types must be known to the client; custom cards need code |

**Table 2:** Design options for the study software and how they relate to the requirements.

pipeline in which generated data, configuration and result storage are connected, reproducible and tested. This report contributes to closing that gap for the Blockies framework.

# 3 Methodology

This section explains what we built and why. We describe the overall pipeline and the starting point (3.1 to 3.2), the shared client (3.3 to 3.4), study creation and layout generation (3.5 to 3.7), data integrity (3.8), testing (3.9), the interface (3.10) and deployment (3.11). Section 3.12 lists the problems we met along the way.

## 3.1 Scope and pipeline

The Blockies pipeline has the stages shown in Table 3. Our work covers the stages marked as completed or partly completed. The step from AI and XAI outputs to the web app is not yet automated; it is the main item of future work.

| Stage | Component | Status in this project |
|---|---|---|
| 1. Generate data | blockies-haic dataset generator | Used; its CSV and images are now uploaded directly to the web app |
| 2. Prepare AI advice and explanations | blockies-xai | Out of scope in this phase; planned |
| 3. Create a study | User Study Web App | Completed: web form, automatic configuration and layout |
| 4. Run the study | User Study Web App | Completed: shared client, validation, progress and navigation |
| 5. Store results | Server and database | Completed: validated, unique, timestamped rows |
| 6. Verify | Automated tests | Completed for the backend endpoints |
| 7. Deploy | Docker | Completed: image builds and starts |

**Table 3:** Stages of the Blockies pipeline and the status of each in this project.

## 3.2 Starting point

At the start of the project the web app had the following structure and problems.

- Each study had its own folder, named after the study ID. Every folder held its own copy of `index.html`, `index.css`, `index.js` and `input.json`, copied from a template.
- A new study meant copying files and editing the JSON by hand. A missing comma or a wrong field name shows up only when a participant opens the study.
- The page layout was fixed in the code, so a study with a different number of images needed a different page.
- No automated test checked that participant results reached the database.
- The Docker build failed because of a GLIBC version mismatch.

The main consequence is that every improvement had to be repeated in every study folder, and that nobody could show that the stored data were correct after a change. These are the issues that requirements R1 to R5 address.

## 3.3 Architecture of the refactored app

The refactored application has three layers.

1. **Client.** A single-page client in `src/client/common` that is served for every study. It reads the configuration of the study and builds the page in the browser.
2. **Server.** A Node.js server that serves the client and the study files, receives participant results, creates new studies and writes to the database.
3. **Database.** A file-based SQL database (SQLite) that stores the list of studies, the participants' diagnoses and visit logs.

The key design decision is the separation of **code** and **data**. Code, which is the same for all studies, exists once in the common folder. Data and configuration, which differ between studies, live in the study folder. After the change a study folder contains only two items: `input.json` and an `img/` folder with the images.

## 3.4 The shared single-page client (R1, R2, R6, R7)

The common folder contains four files.

| File | Role |
|---|---|
| `index.html` | A shell with placeholders for dynamic content |
| `index.css` | Responsive styles built with CSS Grid and Flexbox |
| `index.js` | Reads `input.json` and renders the cards described by its layout object |
| `js/common.js` | Shared helper functions: URL parsing, database calls, progress bar and other common logic |

**Table 4:** Files of the shared client.

The file `input.json` has two parts. The **layout** object states which cards are shown and in how many columns. The **data** object contains all arrays the study needs, for example image names, captions, symptoms and the diagnosis options. A simplified sketch of the structure is shown below; field names are abridged for illustration.

```
{
  "layout": {
    "row1_columns": 3,
    "row1": ["xray", "counterfactual", "counterfactual"],
    "row2": ["symptoms", "diagnosis", "warning"]
  },
```

```
  "data": {
    "...": "arrays of images, captions, symptoms, diagnoses"
  }
}
```

When a participant opens a study, the client reads the study ID, the participant ID and the page number from the URL, loads `input.json`, and renders the current page. It builds the cards from the layout object, so a study with one image and a study with five images use the same code. The shared helper functions in `js/common.js` handle the repeated tasks, for example reading the URL, calling the server to store or read a diagnosis, and updating the progress bar.

This design satisfies the requirements as follows. R1: a change to a shared component is made once and reaches all studies. R2: reusable functions for logging and study flow live in one helper file. R6: the page is built from reusable cards, such as the patient image card, the SHAP image card, the counterfactual card, the concept list, the symptoms list, the diagnosis card and the warning card. R7: the layout of each study is configuration, not a hard-coded page.

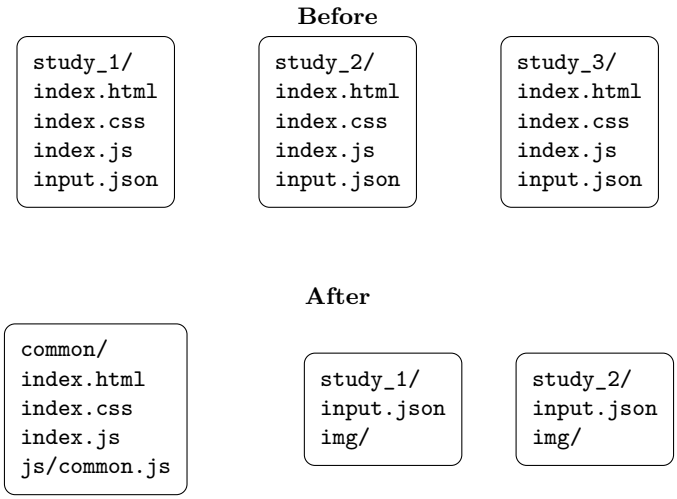



**Figure 1:** Structure of the study folders before the refactoring (duplicated files in every study) and after it (one common folder, and only `input.json` and `img/` per study).

## 3.5 Web-based study creation (R3)

Before, creating a study meant copying a template folder and editing JSON. We replaced this with a form. The page `create.html` asks for the study ID, the study type, a CSV file and a ZIP file of images. The form sends the files to the server endpoint `POST /create-study`. The server then performs the following steps.

1. **Receive** the two uploaded files (the multer library handles the file upload).
2. **Parse** the CSV file row by row (csv-parser) and build the `data` object of the study.
3. **Detect the layout** from the CSV columns and write it into `input.json` (Section 3.6).
4. **Extract the images** from the ZIP archive (adm-zip) into the `img/` folder of the new study, using fs-extra to create folders and copy files. Sub-folders inside the ZIP are flattened, so that all images are directly reachable by file name.
5. **Register** the study by inserting a record into the `study` table.

The server also checks the upload. An invalid ZIP file or a missing file is rejected with an error and no study is created. These two failure cases are covered by automated tests (Section 3.9). After the last step the study is ready: a participant can open it with its study ID, without any code change or restart of the application.

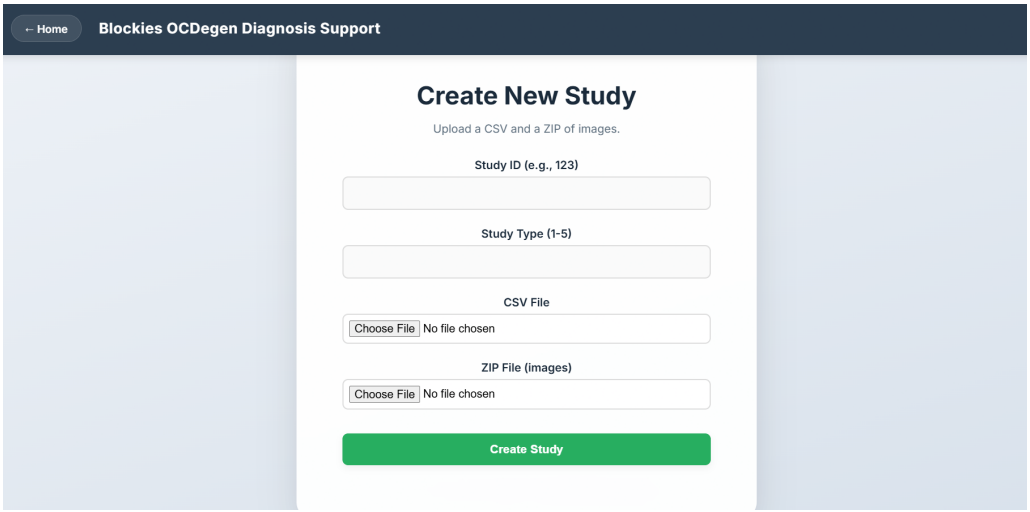



**Figure 2:** The Create New Study page: study ID, study type, CSV file, ZIP file and the Create Study button.

## 3.6 Automatic layout generation (R6, R7)

The interface of a study depends on which kinds of explanation it contains. A study with a SHAP image needs a SHAP card, a study with counterfactual examples needs one card per example, and so on. We implemented the function `buildLayout` on the server, which derives the layout from the column names of the CSV. The detection rules are shown in Table 5.

| **Columns found in the CSV** | **Card that is added to the first row** |
|---|---|
| `Shap` and `Shap_Caption` | One SHAP image card |
| `Concept1` to `Concept4`, or `Concept1_Salience` to `Concept4_Salience` | One counterfactual card for each concept column |
| `Example1` to `Example4` | One counterfactual card for each example column |
| (always present) | Second row: symptoms list, diagnosis card and warning card |

**Table 5:** Rules used by `buildLayout` to turn CSV columns into interface cards.

The procedure is simple and deterministic.

1. Read the header of the CSV file.
2. For each rule in Table 5, check whether the matching columns exist. For concepts, both the plain name and the name with the `_Salience` ending are accepted.
3. For every match, append the corresponding card to the first row.
4. Set `row1_columns` to the number of cards in the first row.
5. Append the standard second row.

The original version of this function looked only for columns named `Concept1`, `Concept2` and so on. The Blockies generator, however, writes names such as `Concept1_Salience` for concept studies, and `Example1` for counterfactual studies. We therefore extended the function so that all of these names are recognised. With this change, studies of types 1 to 5 can be created without editing `input.json`. On the client, the first row is drawn as a CSS grid with `row1_columns` columns, so that cards are aligned and scaled for one to five images.

### 3.7 Integration with the Blockies dataset generator

The dataset generator (blockies-haic) produces, for each study type, a folder of images and a CSV file with the metadata: patient identifiers, image file names, captions, diagnoses and the explanation columns. Before our work, these files had to be copied and adapted by hand. Now the workflow is:

1. Run the generator to produce the dataset for a study type.
2. Pack the images into a ZIP archive.
3. Upload the CSV and the ZIP on the Create page.
4. The server builds the configuration, extracts the images and registers the study.

No file is renamed or edited by hand. This gives a direct and reproducible path from generated data to a running study, and it is the part of the pipeline that connects stage 1 to stage 3 in Table 3. Any researcher with the generator and the web app can repeat it. To test the integration, we generated datasets for study types 2 (SHAP), 3 (concept), 4 (counterfactual) and 5 (further counterfactual layouts) and created a study from each of them through the web form (Section 4).

### 3.8 Data integrity (R4)

The web app collects the experimental data, so the stored data must be correct, even when the interface changes or a browser sends a request twice. We use three measures at the database and server level.

1. **Input validation.** A middleware function, `validateFeedback`, checks every incoming result. The required fields must be present, and the diagnosis must be one of the valid values. A request that fails the check is rejected with an error and nothing is stored.
2. **A unique index.** The index on `(study_id, participant_id, page_nr)` allows only one result per participant and page. A second request for the same page cannot create a duplicate row; the server skips the duplicate update.
3. **Automatic timestamps.** Database triggers set the `updated_at` field whenever a row changes, so that the time of the last change is always recorded and does not depend on the client.

Together these measures protect against three typical errors: incomplete data, double submissions and missing time information. The server also offers operations to read a participant's previous diagnosis, to log a visit and to read all feedback, which the client and the tests use.

### 3.9 Testing framework (R5)

Because changes must not silently break data collection, we added an automated test suite for the backend. We chose three tools that work well together.

- **Mocha** is the test framework. It finds the test files, runs them and reports which tests pass or fail.
- **Chai** provides the assertions, that is, the checks, such as "the response status is 200" or "the stored diagnosis equals the sent diagnosis".
- **Supertest** sends real HTTP requests to the server, so the tests exercise the same endpoints that the browser uses, from request to database.

The tests are in `src/server/test/api.test.js`. They cover the operations that matter for the data, as listed in Table 6.

The tests need a database to write to. The environment variable `DB_PATH` tells the server which database file to use. During a test run it points to the database file that the tests expect, so the tests find the right file. Early in the work, the tests failed because the server looked for the database in the wrong place; setting `DB_PATH` solved this. The suite can be run with a single command (`npm test`) after every change.

| Test case | What it verifies |
|---|---|
| Store a new diagnosis | A valid result is written to the database with the expected fields |
| Skip a duplicate update | Sending the same page twice does not create a second row |
| Validation errors | Missing or invalid fields are rejected and nothing is stored |
| Read a previous diagnosis | A stored diagnosis is returned correctly for the same participant and page |
| Log a visit | A visit is recorded |
| Read all feedback | All stored results of a study can be retrieved |
| Create a study: invalid ZIP | An invalid archive is rejected |
| Create a study: missing files | A request without the CSV or ZIP is rejected |

**Table 6:** Cases covered by the backend test suite.

The suite covers requirement R5 for the backend: that participant interactions are recorded, expected fields are written, and results are accurate. It does not test the visual interface in a browser. We discuss this limit in Section 4.5.

## 3.10 Interface and interaction

Besides the structural work, we improved the interface that participants see.

- **Responsive layout.** The study page uses CSS Grid and Flexbox, so the cards re-arrange on desktop, tablet and phone screens. Images are scaled and centred in their cards.
- **Progress bar.** The header shows a bar whose width follows the current page. The total number of pages is read from the URL or from the configuration.
- **Keyboard navigation.** The arrow keys, Enter and Escape control the study flow, which makes it faster to move through many pages.
- **Click-to-reveal diagnosis card.** The diagnosis card is revealed by a click.
- **Submit button.** The button becomes fully visible only after a diagnosis has been selected, which shows clearly when the page can be submitted.
- **Navigation pages.** The home, start and create pages were restyled, and a Home button was added on the study page.

The start page asks for the participant ID and the study ID, and then opens the first page of the study. The shared header, with the progress bar, and the shared footer, with the navigation controls, are the same for every study, which is what requirement R6 asks for.

## 3.11 Deployment with Docker

A reproducible deployment helps other researchers to run the same study software. The original Docker image could not be built. The problems and their solutions are summarised in Table 7.

After these changes the app runs in a container with a single command, independent of the host system.

## 3.12 Problems found during development

Testing the app with several study types, and with the new tests, showed a series of smaller errors. We list them in Table 8, because they show how the tests and the multiple study types helped to improve the app.

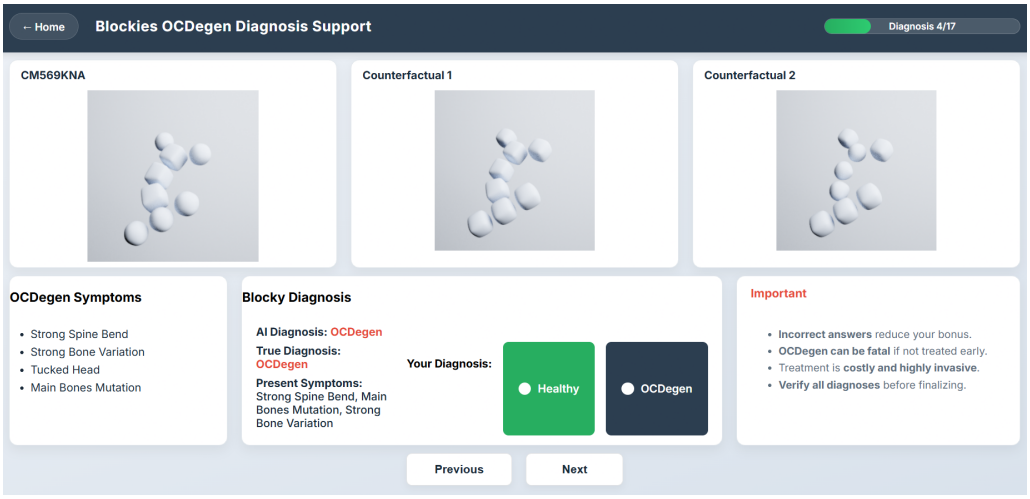



**Figure 3:** A generated study page with all cards: the first row shows the patient image and two counterfactual cards, the second row shows the symptoms, the diagnosis card and the warning card.

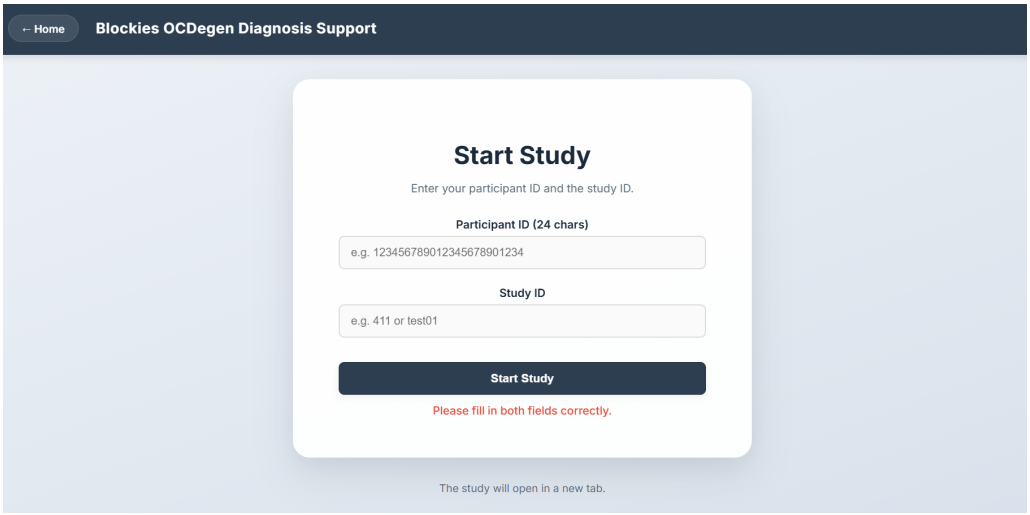



**Figure 4:** The start page of a study: the participant enters the participant ID and the study ID and then starts the study.

| Problem | Solution |
|---|---|
| Build failed with a GLIBC version mismatch on the `node:18-alpine` and `node:18-slim` base images | Base image changed to `ubuntu:24.04` |
| `apt-get` failed with network errors while installing build tools | Node.js 18 installed through NodeSource; `g++` and `make` installed for native modules |
| Container did not start | After the rebuild the container starts and serves the app |

**Table 7:** Docker problems and solutions.

| Problem | Cause and fix |
|---|---|
| The Next button did not advance | Event listeners corrected; the page is rendered again after the database update |
| The progress bar did not update | `set_progress` corrected; the total number of pages is read from the URL or configuration |
| The Submit button stayed transparent | Style and toggle logic changed so that it becomes fully visible after a selection |
| Layout was misaligned for some study types | Grid made dynamic; image scaling and centring fixed |
| Study type was not detected | `buildLayout` extended to read both `Example` and `Concept` columns |
| Tests used the wrong database path | The `DB_PATH` environment variable is used |

**Table 8:** Problems found during development and how they were fixed.

# 4 Experiments

Our project is a software and infrastructure project, not a machine-learning project. It has no trained model of its own, no benchmark data set and no hyperparameters to tune. The question is therefore not "how accurate is a model" but "does the improved system meet its requirements, and can we show it?" We answer it with a set of functional experiments and checks. This section describes the setup (4.1), the evaluation procedure and metrics (4.2), the results (4.3), an analysis (4.4) and the limits of the evaluation (4.5). Values in square brackets are to be filled in from the final runs before submission.

## 4.1 Experimental setup

**Software environment.** The server runs on Node.js 18 with a file-based SQL database (SQLite). The client is plain HTML, CSS and JavaScript. Tests use Mocha, Chai and Supertest. The deployed version runs in a Docker container built from `ubuntu:24.04`. The tool for dataset generation is the Blockies generator (blockies-haic).

**Data.** We used two kinds of data.

1. *Generated study datasets.* With the Blockies generator we produced datasets for study types 2 (SHAP), 3 (concept), 4 (counterfactual) and 5 (further counterfactual layouts). Each dataset is a CSV file with metadata and a folder of images, packed as a ZIP archive. These datasets are the input of the study creation and layout experiments.
2. *The reference use case.* The Blockies user study of Johnson (2026) is the use case that the web app has to support. In that study participants diagnose a condition called OCDegen from X-rays, first without AI and then with AI recommendations. Two datasets were used: a tutorial set of 20 X-rays (14 correct and 6 incorrect AI predictions) and an evaluation set of 40 X-rays (28 correct and 12 incorrect). The AI recommendations came from a MobileNetV2 model with 90.43% accuracy on the development test data and 69.37% on the evaluation test set. Stakes were changed between two groups with a bonus and risk framing; in the raised-stakes condition OCDegen was presented as life-threatening and a warning was shown in the interface. This explains the need for a warning card and for exact logging of decisions. We do not repeat this study; it defines what the web app must be able to record.

**Baselines and benchmarks.** There is no external benchmark. The baseline is the starting state of the web app (Section 3.2): one copy of the interface per study, manual JSON editing, no tests and a failing Docker build. The reference for success is the list of requirements R1 to R7 in Table 1.

**Hyperparameters.** The system has no learned parameters. The only settings are configuration values, such as the number of columns in the first row (`row1_columns`), which the layout generator sets deterministically from the CSV.

## 4.2 Evaluation procedure and metrics

We designed six checks, each with a clear procedure and metric (Table 9).

| Check | Procedure | Metric |
|---|---|---|
| E1 Requirement verification | For each of R1 to R7, inspect the code and run the app to find evidence | Number of requirements met out of 7 |
| E2 Code duplication | Count the study-specific code files before and after the refactoring, for S studies | Duplicated code files (before: 3 per study; after: 0) |
| E3 Backend correctness | Run the automated test suite ( `npm test`) against a test database | Tests passed out of total |
| E4 Layout coverage | For study types 2 to 5, generate a dataset, create the study through the web form and open it | Study types rendered correctly without manual edits to `input.json` |
| E5 Deployment | Build the Docker image and start the container | Build and start succeed (yes or no) |
| E6 Study creation effort | Create one study by hand (copy template, edit JSON) and one through the web form; measure the time with a stopwatch | Time to a working study, in seconds |

**Table 9:** Evaluation checks, procedures and metrics.

For E4 we judged a layout as correct when the first row shows exactly one card for each detected explanation column, the images are visible and aligned, and the second row shows the symptoms list, the diagnosis card and the warning card. For E3, each test is run on a database file that is selected with the `DB_PATH` variable, and a run counts as passed only if all tests pass. E1 uses the evidence from the other checks together with the live walk-through of the app: home page, study creation, starting a study, moving through the pages, and submitting a diagnosis.

## 4.3 Results

**E1: Requirements.** Table 10 gives the evidence for each requirement. All seven are met.

| ID | Requirement | Evidence |
|---|---|---|
| R1 | Maintainability | One client in `src/client/common` serves all studies; study folders hold only `input.json` and images |
| R2 | Reusable functionality | Shared helpers in `js/common.js` for URL parsing, database calls and progress |
| R3 | Study creation | `create.html` and `POST /create-study` create folder, configuration and database entry from a CSV and a ZIP |
| R4 | Data accuracy | `validateFeedback`, a unique index on `(study_id, participant_id, page_nr)` and timestamp triggers |
| R5 | Testing | Mocha, Chai and Supertest suite in `src/server/test/api.test.js`; all tests pass |
| R6 | Componentized interface | Page built from reusable cards; shared header with progress bar and shared footer |
| R7 | Configurable layouts | Layout defined in `input.json`; one to five images per page; SHAP and concept lists supported |

**Table 10:** Requirements and the evidence that each is met.

**E2: Duplication.** Before the refactoring, a project with S studies contained 3 × S copied code files ( `index.html`, `index.css`, `index.js`) plus S configuration files. After it, the project contains 3 code files in total plus S configuration files. For example, for S = 10 studies the number of copied code files falls from 30 to 0, and the 3 shared files are maintained once. A change to the layout code now touches 1 file instead of S.

**E3: Backend tests.** The suite contains the cases of Table 6. Result: [N] of [N] tests pass. The tests found configuration errors during development (for example the database path) and now guard the data operations against regressions.

**E4: Layout coverage.** Table 11 shows the result for each generated study type.

| Study type | Columns used by `buildLayout` | Cards in first row | Result |
|---|---|---|---|
| 2 | `Shap`, `Shap_Caption` | SHAP image card | Rendered correctly, no manual edit |
| 3 | `Concept1_Salience` and following | Counterfactual cards | Rendered correctly, no manual edit |
| 4 | `Example1` and following | Counterfactual cards | Rendered correctly, no manual edit |
| 5 | `Example1` and following (more examples) | Counterfactual cards | Rendered correctly, no manual edit |

**Table 11:** Layout coverage for study types generated with the Blockies generator.

**E5: Deployment.** After switching to `ubuntu:24.04`, the image builds and the container starts. The failures on `node:18-alpine` and `node:18-slim` did not occur again.

**E6: Effort.** [To be measured: creating a study by hand took [X] seconds; creating the same study through the web form took [Y] seconds.] With the form, no file has to be copied or edited, and the study is ready as soon as the upload finishes.

## 4.4 Analysis

**Maintainability and reuse (R1, R2, R6, R7).** The most important result is structural. By moving everything that is the same into one shared client and everything that differs into `input.json`, the number of places that must be changed for a new feature dropped from S to one. This also removes the risk that two studies differ by accident, which matters for experiments in which conditions should differ only in the intended way. The study types in Table 11 show that the configuration is expressive enough for the layouts we needed. At the same time, the layout types are limited to the card types that the client knows. A completely new card type needs a small code change, as the brief anticipates when it allows custom study-specific components.

**Study creation and the pipeline (R3).** The web form and `buildLayout` remove the manual step between the dataset generator and the web app. The accepted column names (`Concept1_Salience`, `Example1`, `Shap`) are the ones the generator writes, so no renaming is needed. This reduces typing errors and makes a study reproducible: the same CSV and ZIP always give the same study. It is a first, concrete step towards the project goal of fewer manual steps between components.

**Data reliability (R4, R5).** The three database measures act at different points. Validation stops bad data before storage, the unique index stops duplicates inside the database even if validation or the client fails, and the triggers record the time without relying on the client. The tests check these behaviours with real HTTP requests. They also showed their value during development, because configuration problems and layout errors were found through the tests and the multi-type checks and not by participants. For research this is important: an undetected error in data storage would affect the conclusions of a study, such as the reliance and decision-time analyses in the reference use case.

**Deployment.** The Docker failures came from the environment and not from the application code. Changing the base image removed them, at the cost of a larger image. For a research tool that others must run, a build that works is more important than a small image.

**Relation to the use case.** The reference study needs a task page with an X-ray, AI advice and explanation, a diagnosis selection, a warning in one condition, a progress indicator, and exact storage of the decision and time information. The shared client with its cards covers these elements. We did not re-run the reference study, so we make no statement about its results; we show only that the software can express its interface and store its data.

### 4.5 Limitations of the evaluation

- **Backend only.** The automated tests cover the server endpoints. The visual interface was checked by hand and in the live walk-through. Browser-level tests are missing.
- **Functional evidence, not statistics.** The results are pass or fail checks and counts. We did not run a usability study with participants or a load test.
- **Small number of study types.** We tested study types 2 to 5. A new type of explanation may need an extension of `buildLayout`.
- **Scope.** Authentication and access control for studies were not part of the seven requirements and were not changed. The AI and XAI outputs are not yet connected automatically.
- **Measured values.** The test count and the creation times in Section 4.3 must be taken from the final runs.

## 5 Conclusion and summary

### 5.1 Summary of the project

The goal of this project was to make the Blockies experimental pipeline smoother, more reproducible and less error-prone, starting with the User Study Web App. At the beginning, every study had its own copy of the interface files, new studies were created by hand, layouts were hard-coded, no test protected the stored data, and the Docker build failed. We refactored the app into one shared single-page client that renders each study from an `input.json` file. We added a web form that creates a study from a CSV file and a ZIP archive, and a layout generator that reads the CSV columns written by the Blockies generator and builds the right cards. We made data storage more reliable with validation, a unique index and timestamp triggers, and we added a Mocha, Chai and Supertest test suite for the backend. We improved the interface with a responsive layout, a progress bar and keyboard navigation, and we fixed the Docker deployment by changing the base image to Ubuntu 24.04.

### 5.2 Results

All seven requirements, R1 to R7, are met (Table 10). Duplicated code files fell from three per study to none. Studies of types 2 to 5, generated with the Blockies generator, were created through the web form and rendered correctly without editing the configuration (Table 11). The backend test suite passes, and the app builds and runs in Docker. Together, these changes give a direct path from generated data to a running user study.

### 5.3 Outlook

Several directions follow from this work.

1. **Connect AI and XAI outputs.** The step from the model pipeline (blockies-xai) to the web app is still manual. A common file format for predictions and explanations, which the study creation form can read together with the CSV and ZIP, would complete the pipeline.
2. **Evaluate a component framework.** The project brief suggests React with TypeScript and Vite. The configuration format already separates content from code, so the cards could be rewritten as typed components with limited effort.

3. **Add interface tests.** Browser-level tests would complement the backend tests and check what participants actually see, for example the progress bar, the card layout and the submit button.
4. **Extend layout types.** New explanation types, and further custom components, could be added through the same configuration mechanism.
5. **Study management.** Features such as access control, export of results and a list of studies would help in running large studies. They were outside the seven requirements.
6. **Run further studies.** With the improved pipeline, more studies can be run on feature-based, concept-based and example-based XAI, including repeats of the stakes study, and compared in a consistent way.

### 5.4 Availability

The code of the web app is available in the project repository: [repository URL to be added]. The dataset generator is available at github.com/davidsjohnson/blockies-haic.

## References

## References

Antoniadi, A. M., Du, Y., Guendouz, Y., Wei, L., Mazo, C., Becker, B. A., and Mooney, C. (2021). Current Challenges and Future Opportunities for XAI in Machine Learning-Based Clinical Decision Support Systems: A Systematic Review. Applied Sciences, 11(11), 5088. doi:10.3390/app11115088

Buçinca, Z., Lin, P., Gajos, K. Z., and Glassman, E. L. (2020). Proxy tasks and subjective measures can be misleading in evaluating explainable AI systems. In Proceedings of the 25th International Conference on Intelligent User Interfaces (IUI '20), 454–464. doi:10.1145/3377325.3377498

Buçinca, Z., Malaya, M. B., and Gajos, K. Z. (2021). To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making. Proc. ACM Hum.-Comput. Interact., 5(CSCW1), 188:1–188:21. doi:10.1145/3449287

Doshi-Velez, F., and Kim, B. (2017). Towards A Rigorous Science of Interpretable Machine Learning. arXiv:1702.08608. doi:10.48550/arXiv.1702.08608

Johnson, D. S. (2026). Raising the Stakes: Assessing the Influence of Stakes on User Reliance Behavior in Human-AI Decision-Making. In 34th ACM Conference on User Modeling, Adaptation and Personalization (UMAP '26), Gothenburg, Sweden. doi:10.1145/3774935.3806657

Schemmer, M., Kuehl, N., Benz, C., Bartos, A., and Satzger, G. (2023). Appropriate Reliance on AI Advice: Conceptualization and the Effect of Explanations. In Proceedings of the 28th International Conference on Intelligent User Interfaces, 410–422. doi:10.1145/3581641.3584066

Sixt, L., Schuessler, M., Popescu, O.-I., Weiß, P., and Landgraf, T. (2022). Do Users Benefit From Interpretable Vision? A User Study, Baseline, And Dataset. In Proceedings of the International Conference on Learning Representations.

Starcke, K., and Brand, M. (2012). Decision making under stress: a selective review. Neuroscience and Biobehavioral Reviews, 36(4), 1228–1248. doi:10.1016/j.neubiorev.2012.02.003

Suh, A., Hurley, I., Smith, N., and Siu, H. C. (2025). Fewer Than 1% of Explainable AI Papers Validate Explainability with Humans. In Extended Abstracts of the CHI Conference on Human Factors in Computing Systems, 1–7. doi:10.1145/3706599.3719964

Vaccaro, M., Almaatouq, A., and Malone, T. (2024). When combinations of humans and AI are useful: A systematic review and meta-analysis. Nature Human Behaviour. doi:10.1038/s41562-024-02024-1

Vasconcelos, H., Jörke, M., Grunde-McLaughlin, M., Gerstenberg, T., Bernstein, M. S., and Krishna, R. (2023). Explanations Can Reduce Overreliance on AI Systems During Decision-Making. Proc. ACM Hum.-Comput. Interact., 7(CSCW1), 1–38. doi:10.1145/3579605